

Stocker — an AI Investment Team on n8n

Final project · Track 1 — Financial Automation: a Multi-Agent n8n System

[Student name(s) & ID(s) — fill in before submission] · Repository: github.com/gilbarell1/algotrade-project
· July 2026

1 · The financial problem

Forming a view on a stock means synthesizing three unlike sources: price action, news flow, and exchange disclosures. Doing that by hand across a watchlist is slow and inconsistent. Doing it with a single LLM prompt is fast but untrustworthy — one-shot pipelines invent financial figures, hide their uncertainty, and overcommit on thin evidence.

Tel-Aviv names add specific friction. Coverage splits across Hebrew and English, and TASE disclosures live on a bot-protected JavaScript site whose actual figures sit in PDF attachments two layers below the page a naive scraper would read.

What this project solves. Given a watchlist that may mix TA-35 and S&P 500 names, produce a justified **long / short / hold / avoid** recommendation per ticker — with a conviction level, a complete reasoning trace, and citations back to every source. Two guarantees hold throughout: **no financial figure is ever fabricated**, and **no uncertainty is ever hidden**. A failed data source lowers conviction; it never becomes a signal.

2 · What was built

A virtual investment team of four AI agents orchestrated by **n8n**, backed by a local **FastAPI** service that owns all real computation. Three specialists gather independent evidence per ticker; a Risk Manager weighs it through a three-pass self-critique and issues the call. Every run produces a **PDF report** with an executive summary, per-ticker pages, and the full reasoning trace.

Agent	Role	Key mechanism
Sentiment claude-haiku-4.5	Reads recent news from NewsAPI and RSS, English and Hebrew	Dual scoring. An LLM and a fine-tuned transformer (FinBERT for English, HeBERT for Hebrew) score every article independently. Their disagreement is itself a reported signal.
Earnings grok-4.3	Reads exchange disclosures — Maya for TASE, SEC EDGAR for US, routed by market	Self-consistency. Every figure is sampled three times at temperature 0.3 and committed only on a majority vote of verbatim matches. Anything else is marked ambiguous .
Technical gemini-2.5-flash-lite	Narrates RSI, MACD, Bollinger and ATR computed server-side	The model narrates pre-computed JSON only. No number in the report originates inside a language model.
Risk Manager claude-haiku-4.5, ×3	Synthesizes the three signals into the final call	

Draft → devil's-advocate critique → final, then a deterministic rubric clamp so a model that breaks the decision rules cannot reach the report.

Three entry points share one pipeline. A **manual trigger** for demos. An **hourly schedule** with a per-market gate — a run at 11:00 Israel time analyzes only TASE names, one at 20:00 only US names, each market carrying its own calendar, hours, news feeds, earnings source and currency. And a **chat assistant** with a web UI: “*what do you think about Teva?*” runs the real pipeline for **TEVA.TA** in roughly a minute and answers with the Risk Manager's call. The chat agent is deliberately a **router, not an analyst** — it may only invoke the pipeline and relay what comes back, and it declines to produce any figure the tools did not return.

3 · Architecture

Two layers joined by one HTTP boundary. n8n holds the orchestration and every LLM call, and contains no machine-learning code. The FastAPI service owns everything needing a real library — indicators (**pandas-ta**), transformer inference, PDF rendering (WeasyPrint over Jinja2), and scraping (Playwright for Maya's protected SPA, plain HTTP for EDGAR) — persisting to **DuckDB**: prices, news, earnings, runs, recommendations, costs.

Each run opens with a run id and the watchlist, fans the three analysis agents out per ticker at concurrency 3, runs the Risk Manager's three passes per ticker, harvests real token usage from n8n's execution API, renders the PDF, and closes the run.

- **Heavy data never crosses the LLM boundary.** Price arrays, article bodies and rendered HTML stay server-side; models see short text and scores.
- **Every LLM response is schema-validated.** A malformed reply earns one stricter retry, then an explicit degraded result — never a silent fallback.
- **Failures degrade, they never crash.** Yahoo, NewsAPI, RSS, Maya, EDGAR and the n8n API each fail into a labelled state that the Risk Manager turns into lower conviction.
- **Costs are observable.** Every call is logged with tokens, dollars and latency; a five-ticker run costs roughly **\$0.04–0.08**.
- Times are stored in UTC and rendered in Asia/Jerusalem. Secrets live only in a gitignored **.env**.

4 · The AI, beyond a baseline pipeline

Technique	What it adds
Dual sentiment LLM alongside FinBERT / HeBERT	An independent, non-LLM opinion on every article. Disagreement above a threshold caps conviction at medium and is printed in the report — model conflict becomes information rather than something averaged away.
Self-consistency extraction n = 3, temperature 0.3, majority vote	“Never invent numbers” enforced by construction rather than by instruction. A figure is committed only when repeated samples agree on text present verbatim in the source; everything else prints as ambiguous , visually distinct in the PDF.

Three-pass critique loop with a deterministic rubric clamp	The recommendation visibly stress-tests itself, and all three passes are printed. A mechanical clamp then enforces the decision rubric — and when it overrides the model, the report says so rather than quietly rewriting the model's words.
Versioned few-shot prompts with an evaluation harness	Prompts are files, not workflow internals: version-controlled, reviewable, and scored against hand-labelled fixtures by a single command.
Degraded-agent epistemics	A failed agent is defined as <i>no information</i> . The prompts state that an empty result caused by a system failure says nothing about the company, and the rubric counts it for neither side while still lowering conviction.

5 · Results

5.1 · Evaluation harness

Scored against hand-labelled fixtures: 30 news items (20 English, 10 Hebrew), 10 disclosures, and 7 chat-router cases.

Agent	Result
Sentiment — LLM	accuracy 0.90 (27 / 30) · MAE 0.12
Sentiment — FinBERT / HeBERT	accuracy 0.63 (19 / 30) · MAE 0.41
Sentiment — agreement	Pearson r 0.70 (n = 30)
Earnings — classifier	macro-F1 1.00 · materiality accuracy 0.90 (n = 10)
Earnings — extractor	precision 1.00 · recall 1.00 · ambiguous-when-absent 22 / 22
Chat router	refusal 4 / 5 · routing 2 / 2 · no fabrication

Read honestly, the transformer arm is weaker than the LLM arm largely because **HeBERT is a general Hebrew sentiment model, not a finance-tuned one**. That gap is measured and published rather than hidden — and it is precisely why disagreement between the two arms is treated as a signal instead of noise. The extractor row is the “never invent numbers” guarantee, measured: every figure absent from its source came back **ambiguous**, 22 times out of 22.

5.2 · Live behaviour, including under failure

A single-ticker chat run completes in about a minute for two to four cents, producing a four-page PDF and complete database rows. A mixed watchlist renders one report carrying per-market currencies and a market-grouped summary.

Case study — what happens when a data source dies

With a rejected news API key the sentiment agent degrades to zero articles. In an early run the critique pass argued that the *absence* of news for a mega-cap was “itself a material anomaly... both scenarios bearish” — inferring a market signal from an infrastructure failure. This was caught in review and root-caused: the agent's failure status was not machine-readable to the prompts. After the fix, the same ticker's critique instead notes that the system “correctly avoids inventing sentiment

signal”, and the final call states that *“a degraded agent is neutral by definition... its absence is not a bearish or bullish signal”* — while conviction is still lowered. The failure, the diagnosis and the before/after evidence are all in the repository history.

5.3 · The reasoning trace, as printed in the report

Excerpts from one run's three passes for **NFLX**, after the fix described above. Each pass is printed in full on the ticker's page.

1 · DRAFT

hold · low

“Agreement count is 1 of 3: only earnings aligns with a potential direction, while sentiment is degraded (**NewsAPI failure provides no signal**) and technical is neutral. Per rule 2, one or fewer agents agreeing mandates a ‘hold’ call regardless of conviction cap.”

2 · DEVIL'S-ADVOCATE CRITIQUE

counter: hold

“MACD below signal line with positive histogram — a subtle bullish inflection the draft dismisses as merely neutral... Agreement count is mechanically applied, but the denominator includes a degraded sentiment agent that contributed zero information. **The system correctly avoids inventing sentiment signal**, but the true agreement ratio is understated.”

3 · FINAL DECISION

hold · low

“The critique's technical observations are noted but do not override the mechanical agreement rule... The critique correctly identifies that the degraded sentiment agent weakens the denominator, which legitimately lowers conviction further; however, **per the rubric, a degraded agent is neutral by definition and does not constitute evidence of direction — its absence is not a bearish or bullish signal.**”

Note how the critique attacks the draft on its own evidence, and the final answers each objection explicitly rather than restating the draft. Full report pages and screenshots are in the repository.

6 · Risks and caveats

- **The critique loop reduces overconfidence; it does not guarantee correctness.** It is a structured reasoning aid, not a financial-validity guarantee. The report is educational and not investment advice, and its footer says so.
- **No live execution and no return measurement.** The system argues positions; it never checks whether they would have made money. A backtest is the natural next step.
- **Maya scraping is best-effort.** A protected JavaScript site rendered headless: layout changes can break the harvest. Figures depend on a PDF attachment two layers down, so a scanned file or dead link yields **ambiguous** figures — never guessed ones.
- **EDGAR extraction is a bounded text excerpt, not a structured parse.** A 10-Q's tables flatten badly into text, so press-release exhibits are preferred explicitly by the selection logic.
- **News coverage is uneven.** TA-35 mid-caps are thinly covered, and auto-derived search terms fail where a company's registered name differs from its press name (“Alphabet” versus “Google”, hand-curated). The free news tier binds on a large watchlist.

- **A model swap is not a no-op.** Models clear schema validation at different rates, and the retry path is a different branch of the graph — swapping the earnings model surfaced a latent fan-in bug the previous model had masked.
- **Two known open items.** The agreement rule still counts a degraded agent in its three-agent denominator, so “1 of 3” may effectively be “1 of 2 measurable” — flagged by the system's own critique pass. And one chat-router refusal case out of five still fails.

7 · Future development

- **Backtest the recommendation stream** against realized returns — turning rationale quality into measured alpha, net of costs.
- **Delivery channels.** E-mail and Telegram are each a single additional n8n node; the report step is already structured for them.
- **Scale the watchlist** to the full TA-35 and S&P 500. The per-market architecture supports it today; a paid news tier is the binding constraint.
- **Rubric refinements.** Resolve the degraded-denominator question, and consider a stronger Risk Manager model — a one-field change, with the cost table as arbiter.
- **OCR fallback** for scanned disclosure PDFs, currently an honest *ambiguous*.

Running it. `npm run setup`, add API keys to `.env`, then `npm run dev` to start the quant service, n8n and the chat UI together; the six workflows are imported once through the n8n editor. Quick-start, evaluation reproduction and wiring details are in [README.md](#); the complete technical design in [docs/design.md](#); run walk-throughs in [docs/results.md](#); and the demo outline in [docs/demo_script.md](#).